FOUNDATIONS OF DEEP REINFORCEMENT LEARNING

INTERMEDIATE DEEP LEARNING

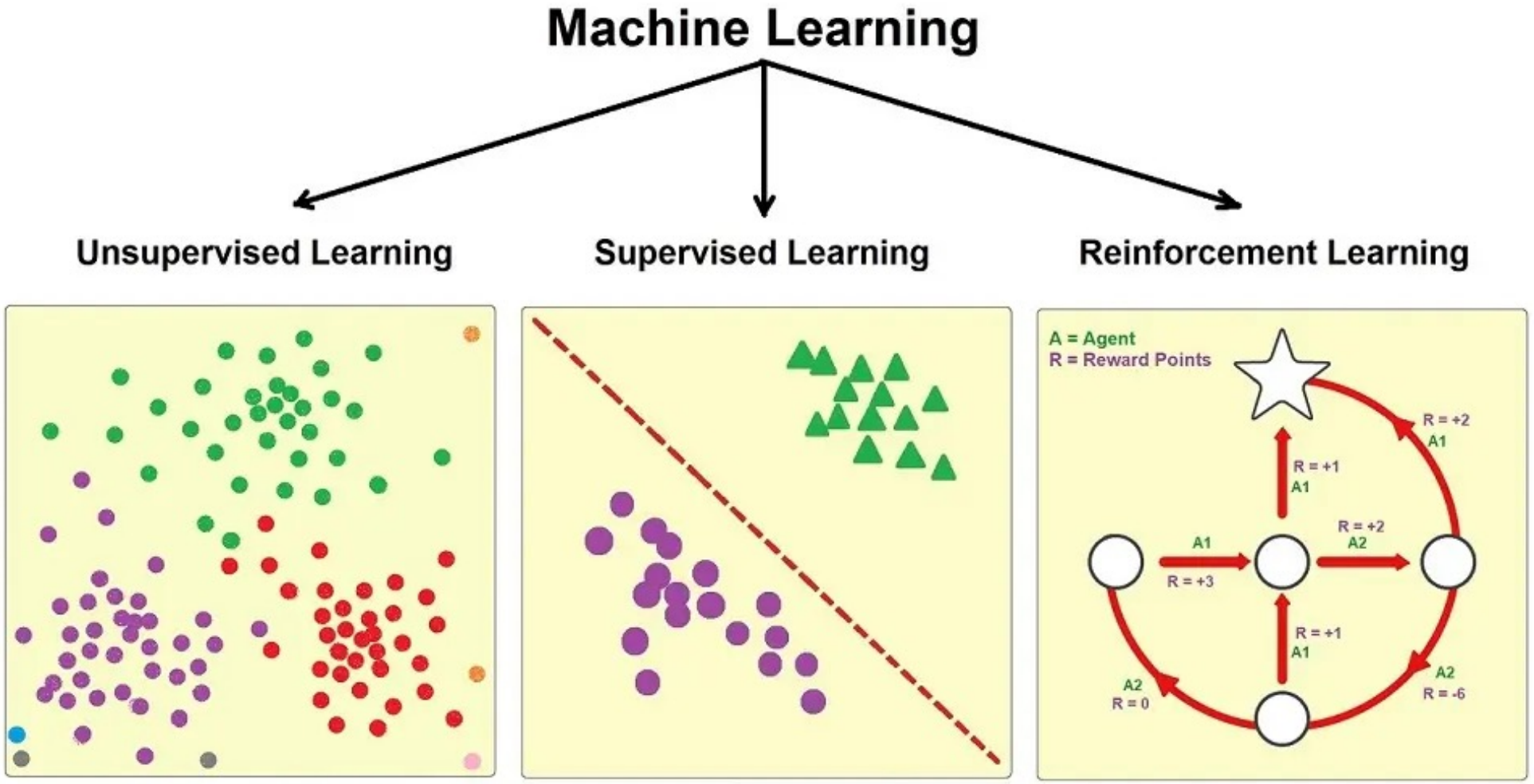
Zhuofan XU
AI / Reinforcement Learning Engineer @ Deepmath
PhD in AI - Université Paris-Saclay · ENS Paris-Saclay

INTRODUCTION

REINFORCEMENT LEARNING – MARKOV DECISION PROCESS – GENERAL CONCEPT

INTRODUCTION

REINFORCEMENT LEARNING



INTRODUCTION

REINFORCEMENT LEARNING



Reinforcement Learning (RL) is concerned with solving sequential decision-making problems

Video games

Sports

Driving

Robotic control

Biology

An objective or goal (to reach or to maximize)

INTRODUCTION

REINFORCEMENT LEARNING

What Humans Do

- To achieve these objectives, we continuously interact with the environment by taking actions.
- We receive feedback from the environment.
- Our actions influence our future states.
- The process may involve inherent randomness.

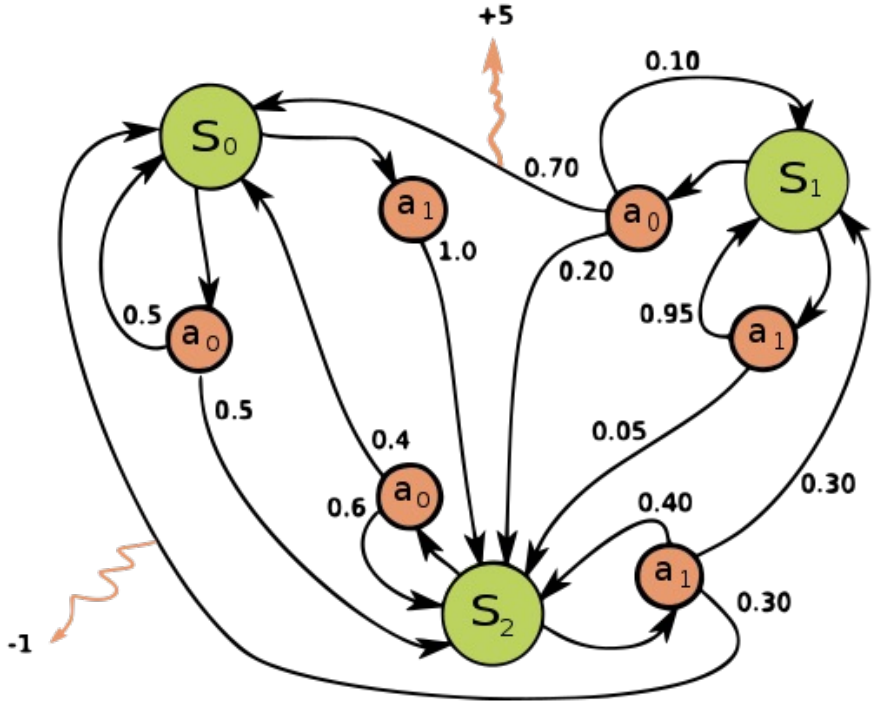
What We Expect Machines to Do

- Develop a strategy for choosing actions based on the environment's state.
- Learn the strategy through interaction with the environment.
- Aim to reach or maximize the objective.
- Be able to handle randomness.

INTRODUCTION

MARKOV DECISION PROCESS

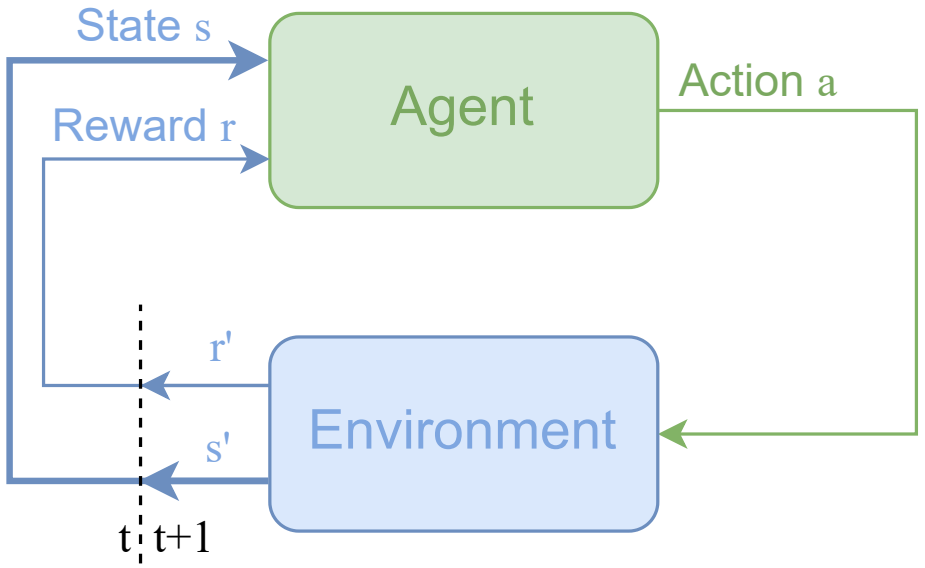
- A Markov Decision Process (MDP)
  - State S
  - Actions A
  - Transition $P(s'|s,a)$
  - Reward $R(s,a)$
- Formal framework used to model decision-making under uncertainty
- Markov property: next state depends only on the current state and action, not on past history $P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}|s_t, a_t)$



INTRODUCTION

GENERAL CONCEPT OF RL

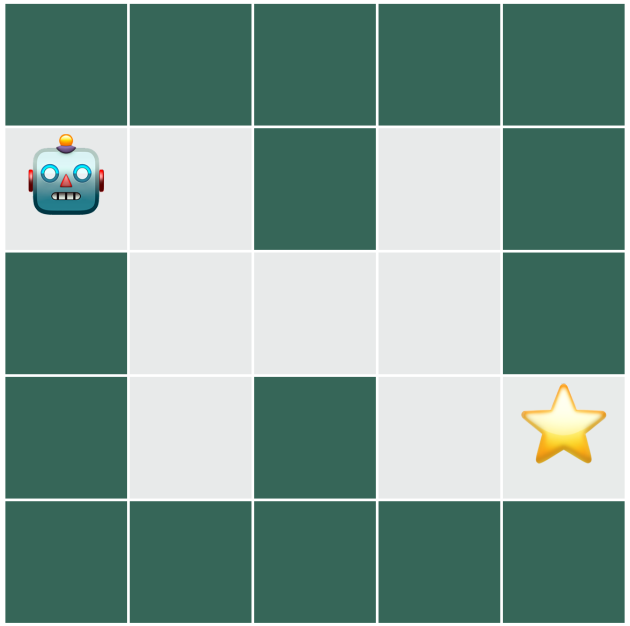
- No predefined dataset
- The agent is free to interact with the environment
- At each time step t, the agent observes the current state s, and selects an action a
- The environment transitions the agent to the next state s', based on s and a, and provides a reward r'
- The agent's goal is to maximize the cumulative reward over an episode (a sequence of time steps)



INTRODUCTION

EXAMPLE: GRID MAZE

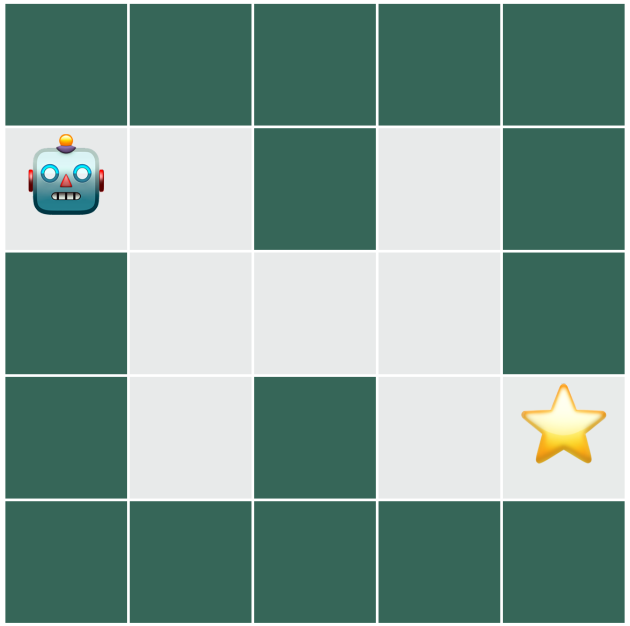
- Environment: Grid maze
- State: Agent's coordinates (r, c); current state $s_0 = (1,0)$
- Actions: $\{\text{Up}, \text{Down}, \text{Left}, \text{Right}\}$
- Reward: -1 per step
- Episode termination: At 100 steps (max), or when the goal is reached



INTRODUCTION
POLICY

- The goal of learning is for the agent to develop a strategy for selecting an action a_t based on the current state s_t.
- This mapping is called a **policy**, denoted as:

$$a_t = \pi(s_t)$$



CAN A POLICY BE LEARNED USING SUPERVISED LEARNING?

In some simple cases, yes — but it's not a universal solution because of:

- No ground-truth labels: In complex environments, even humans don't know the correct action for every state.
- Delayed and sparse feedback.
- Exception: imitation learning: It works when expert demonstrations are available. But…

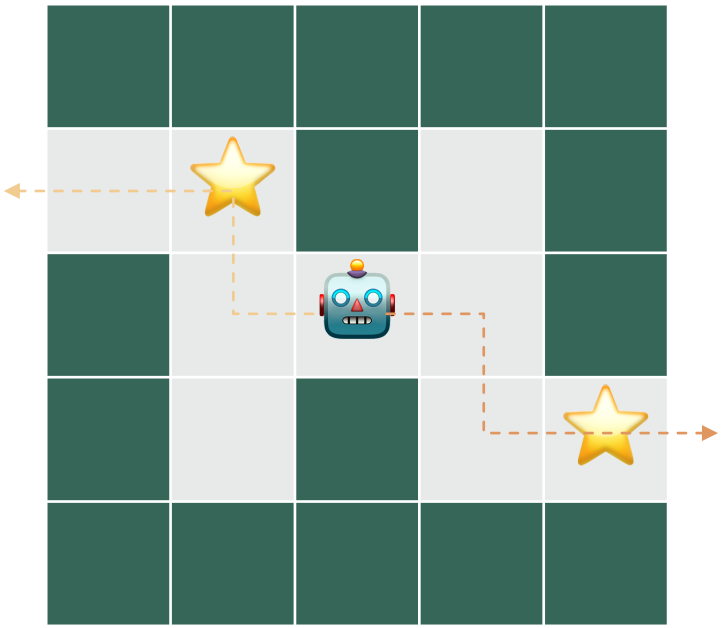


REINFORCEMENT LEARNING AND DEEP REINFORCEMENT LEARNING

Q-LEARNING – DEEP Q-LEARNING

RETURN FUNCTION

- Return G_t is the discounted cumulative reward from current step to the end of episode (T)
- $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$,
- Where $\gamma \in [0,1)$ is the discount factor.
- It balances the preference between immediate reward and future rewards.
- What does it mean if $\gamma = 0$?
- What happens when $\gamma = 1$?



If reward of reaching a star is 1, and reach the exit of the maze gives reward of 5…

Q-LEARNING

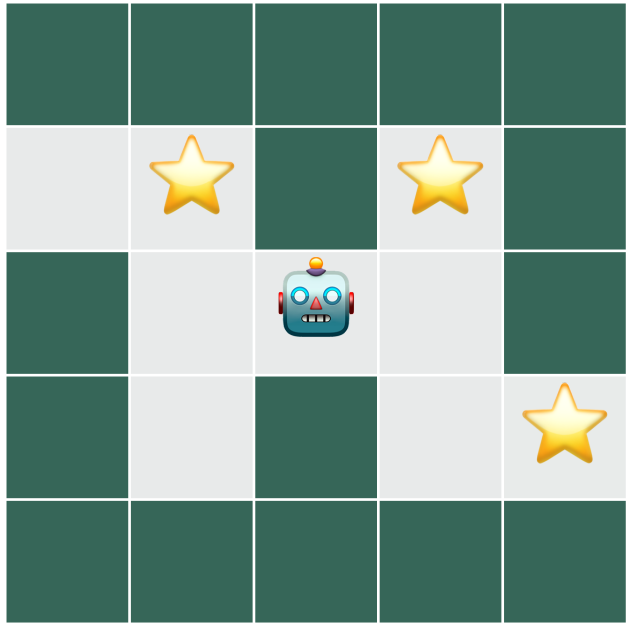
- Value functions help determine the policy.
- Example: Q-Learning
- A function Q calculates the **quality** of each $(s, a) \in S \times A$ pair
  - $Q\colon S \times A \rightarrow \mathbb{R}$
- Policy: at each $s \in S$, choose the $a \in A$ that maximizes the Q-value
  - $\pi(s) = \arg\max_a Q(s, a)$

- A natural candidate for measuring quality is the return G

Q-LEARNING

- $Q(s, a)$ is the expected return from choosing a at state s.

- In the robot's current location, could you calculate the Q-value of the two possible actions $\{\text{left}, \text{right}\}$?

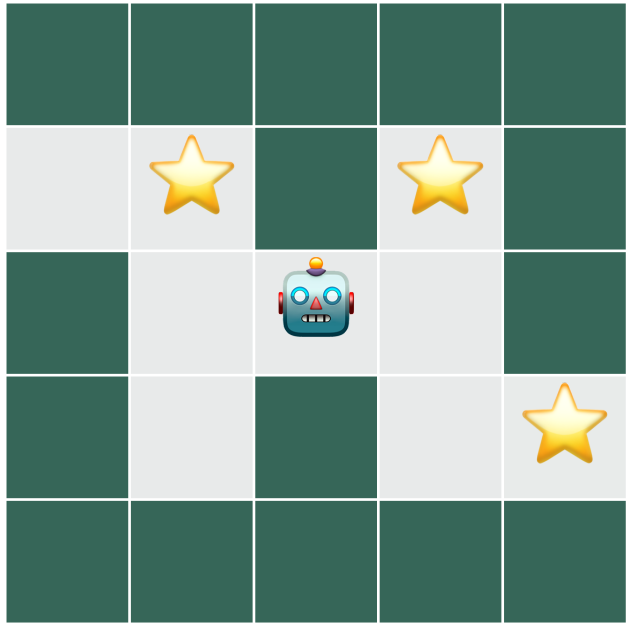


If reward of reaching a star is 1, and reach the exit of the maze gives reward of 5…

RL AND DRL

Q-LEARNING

- In the grid maze example, most states have more than one possible action. If we consider 10 steps, the number of possible trajectories can reach $2^{10} = 1024$. In a game with $T = 100$, the number of trajectories grows exponentially, making it impossible to track the empirical returns.

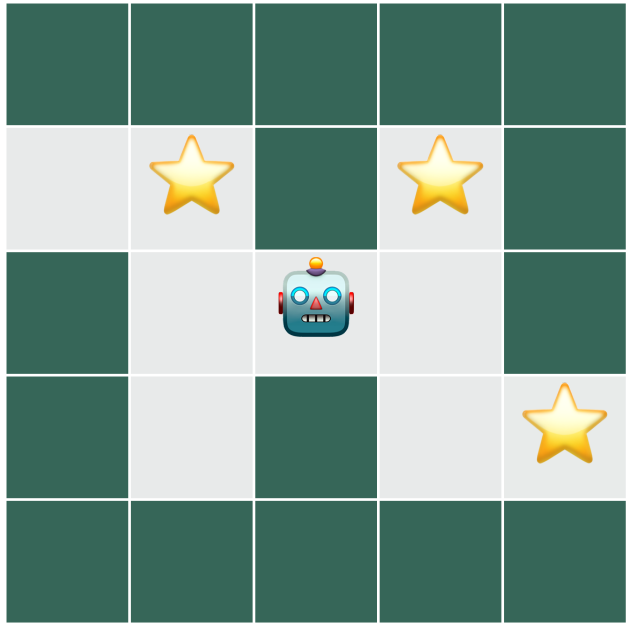


If reward of reaching a star is 1, and reach the exit of the maze gives reward of 5, each time step gives reward of -1

RL AND DRL

Q-LEARNING

- Q-learning solves this problem by:
  - Assuming that the agent always chooses the action with the highest Q-value, and that this Q-value represents the optimal expected return.
  - Using bootstrapping through Temporal Difference (TD) updates to refine the Q-values.



If reward of reaching a star is 1, and reach the exit of the maze gives reward of 5, each time step gives reward of -1

Q-LEARNING: BELLMAN'S EQUATION

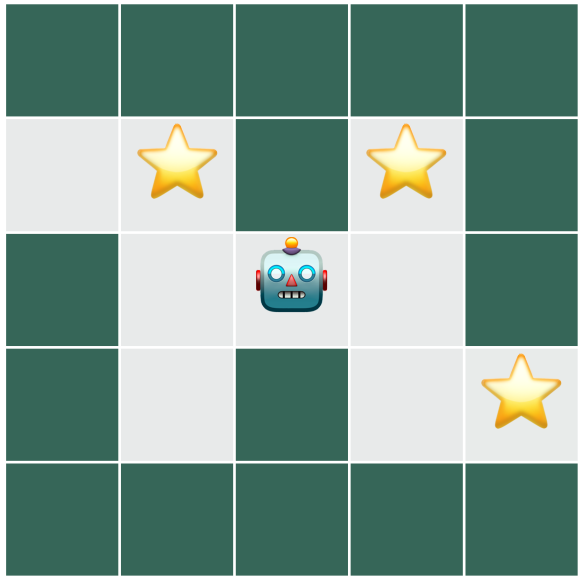
- The Bellman Optimality Equation expresses that the optimal value of a state (or state–action pair) equals its immediate reward plus the discounted value of the best possible next state:
- $Q(s,a) = E\left[r + \gamma \max_{a'} Q\,(s', a')\right]$

Q-LEARNING: UPDATE RULE

- The Bellman equation defines what the optimal Q-values should satisfy, while the Q-learning update rule iteratively applies this relationship using bootstrapping to approximate them:
- $$Q(s,a) \leftarrow Q(s,a) + \alpha \underbrace{[\, r + \gamma \max_{a'} Q\,(s', a') - Q(s,a)\,]}_{\text{TD Error } \delta}$$
- In this formula, the TD (Temporal Difference) which measures the difference between the predicted value and the updated, bootstrapped estimate of future return.

Q-LEARNING: UPDATE RULE

- $Q(s,a) \leftarrow Q(s,a) + \alpha \underbrace{[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]}$

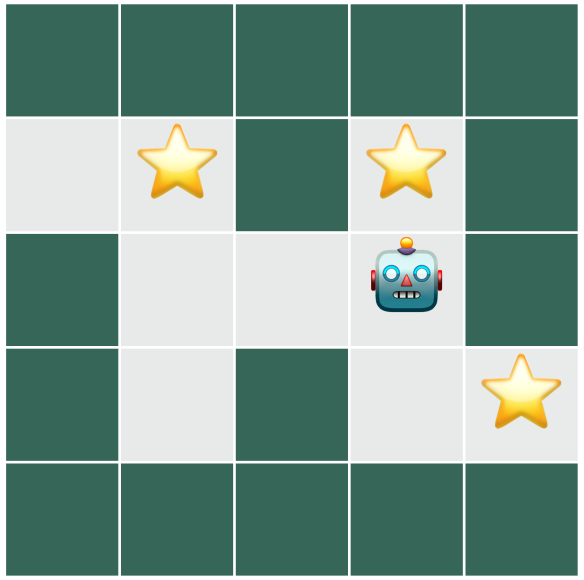


↑	→						
←	↓						
0	0	0	0		0	0	
0	0	0	0		0	0	
		0	0	0	0	0	0
		0	0	0	0	0	0
		0	0		0	0	0	0
		0	0		0	0	0	0

Q-values

Q-LEARNING: UPDATE RULE

- $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\,r + \gamma \max_{a'} Q\,(s', a') - Q(s,a)\,]$



$r = -1$

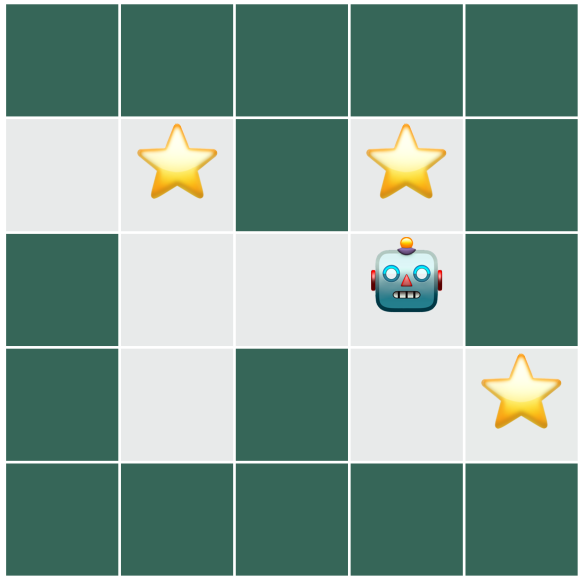
↑	→						
←	↓						
0	0	0	0		0	0	
0	0	0	0		0	0	
		0	0	0	0	0	0
		0	0	0	0	0	0
		0	0		0	0	0	0
		0	0		0	0	0	0

Q-values

RL AND DRL

Q-LEARNING: UPDATE RULE

- $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\,r + \gamma \max_{a'} Q\,(s', a') - Q(s,a)\,]$



$r = -1$

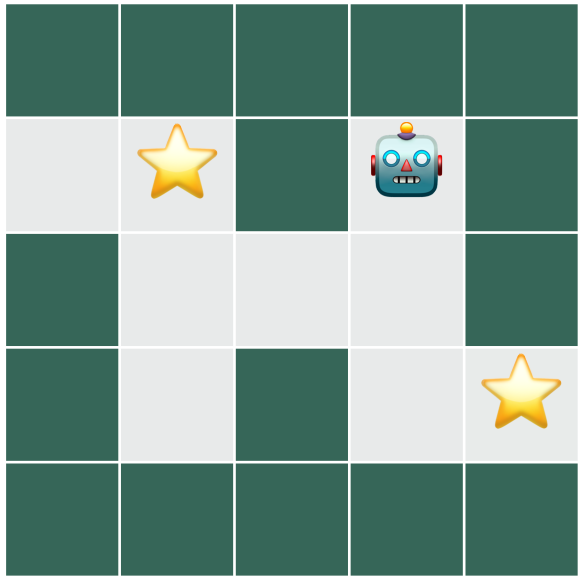
↑	→								
←	↓								
0	0	0	0			0	0		
0	0	0	0			0	0		
		0	0	0	-.5	0	0		
		0	0	0	0	0	0		
		0	0			0	0	0	0
		0	0			0	0	0	0

Q-values

RL AND DRL

Q-LEARNING: UPDATE RULE

- $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\,r + \gamma \max_{a'} Q\,(s', a') - Q(s,a)\,]$



$r = 1$

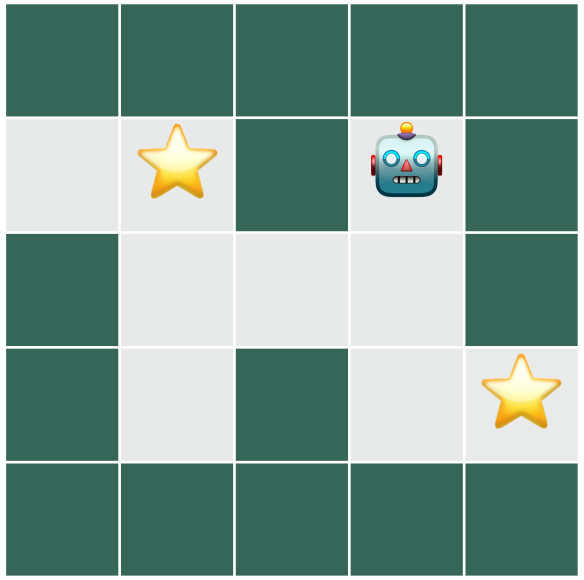
↑	→								
←	↓								
0	0	0	0			0	0		
0	0	0	0			0	0		
		0	0	0	-.5	0	0		
		0	0	0	0	0	0		
		0	0			0	0	0	0
		0	0			0	0	0	0

Q-values

RL AND DRL

Q-LEARNING: UPDATE RULE

- $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\,r + \gamma \max_{a'} Q\,(s', a') - Q(s,a)\,]$



$r = 1$

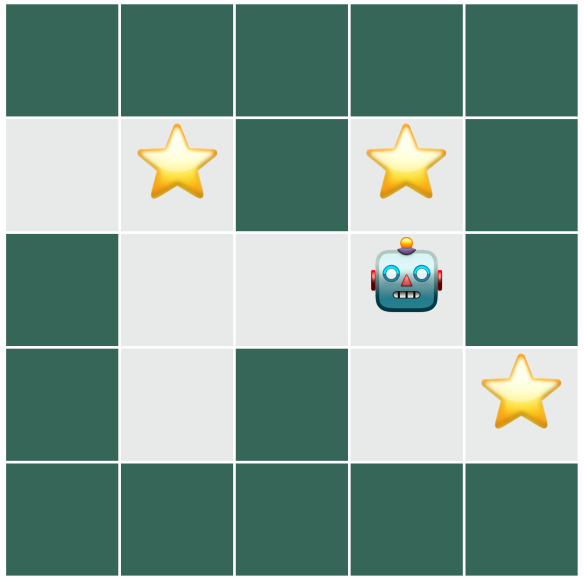
↑	→								
←	↓								
0	0	0	0			0	0		
0	0	0	0			0	0		
		0	0	0	-.5	.5	0		
		0	0	0	0	0	0		
		0	0			0	0	0	0
		0	0			0	0	0	0

Q-values

RL AND DRL

EXPLORATION AND EXPLOITATION

- In this situation, will the agent try other actions?



↑	→								
←	↓								
0	0	0	0			0	0		
0	0	0	0			0	0		
		0	0	0	-.5	.5	0		
		0	0	0	0	0	0		
		0	0			0	0	0	0
		0	0			0	0	0	0

Q-values

EXPLORATION AND EXPLOITATION

- In RL, exploration means trying new actions to discover potentially better rewards, while exploitation means choosing the known best action to maximize immediate reward.
- It is important to explore new actions to avoid getting stuck in a local optimum, but Q-learning does not do this explicitly.
- $\rightarrow$ We enforce an exploration strategy using ϵ-greedy

ϵ-GREEDY

- The ϵ-greedy strategy introduces a hyperparameter ϵ. During training, each time the agent selects an action, it generates a random number i.
  - If $i < \epsilon$, the agent chooses a random action among the possible ones;
  - otherwise, it selects the action with the highest Q-value.
- It is also possible to vary ϵ during training.
  - At the beginning, a higher ϵ encourages exploration of actions,
  - while toward the end, a lower ϵ promotes exploitation of the best actions so their Q-values can converge.
- This approach is known as ε-decay.

Q-LEARNING WITH ϵ-GREEDY

Algorithm 1 Q-Learning Algorithm

Initialize $Q(s, a)$ arbitrarily for all states and actions
for each episode **do**
    Initialize state s
    while episode not done **do**
        Choose action a using ε-greedy policy
        Take action a, observe reward R and next state s'
        $Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a)\right]$
        $s \leftarrow s'$
    end while
    Decay ε (reduce exploration over time)
end for

CODING

TD1: Q-LEARNING

ON POLICY AND OFF POLICY

- For Q-Learning, during training, we usually behave with an exploratory policy:

$$a \sim \mu(\cdot\,|s) \qquad \text{e.g. } \varepsilon\text{−greedy}$$

- But the update assumes the greedy action at the next state:

$$a^* = \arg\max_{a'} Q(s', a')$$

- So we have two different policies:

$$\underbrace{\mu}_{\text{Behavior policy}} \neq \underbrace{\pi}_{\text{Target policy}}$$

- Q-Learning is **off-policy**: it can learn one policy while collecting data with another.

ON POLICY AND OFF POLICY

- What if we want the update to follow the policy we actually execute?
- → On-policy
- Example: SARSA
  - Updates Q-values using the next action actually selected by the current policy

ON POLICY AND OFF POLICY

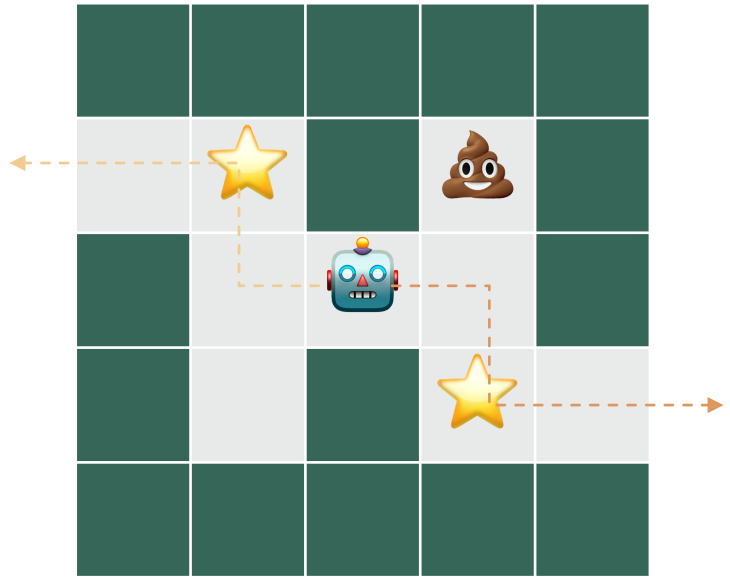
- Target of Q-Learning: $r + \gamma \max_{a'} Q(s', a')$
- Target of SARSA: $r + \gamma Q(s', a')$, where $a' \sim \pi(\cdot \mid s')$
- Update with SARSA:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- SARSA learns from what its current policy actually does

ON POLICY AND OFF POLICY

- SARSA Learns the consequences of the actual ϵ-greedy policy
- In the situation on the left, which path will each algorithm learn to prefer?
  - Q-Learning?
  - SARSA?



💩 = −10, ⭐ = +1

DEEP REINFORCEMENT LEARNING (DRL)

- To address complex and high dimensional problems
- Neural networks as policy function or value function
- Example:
  - Tabular Q Learning → Deep Q Network (DQN)
- Better generalization

Q-Table	a_1	a_2	a_3	...	a_m
s_1					
s_2					
s_3					
s_4					
...					
s_n					

Figure : Q Table

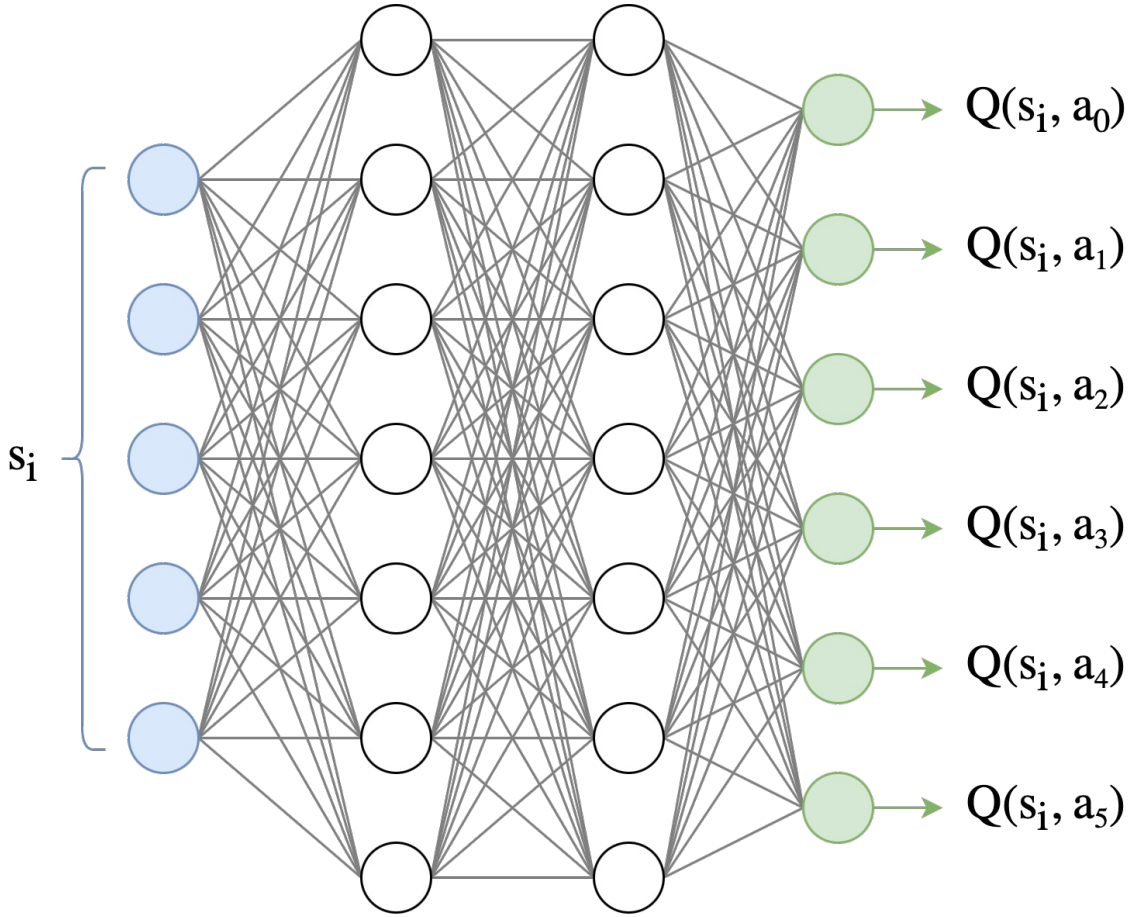


Figure : Q Network

CODING

TD2: SIMPLE DQN